

Image Segmentation

Pablo Mesejo

pmesejo@go.ugr.es

Universidad de Granada

Departamento de Ciencias de la Computación e Inteligencia Artificial



UNIVERSIDAD
DE GRANADA



DaSCI

Instituto Andaluz de Investigación en
Data Science and Computational Intelligence

Readings

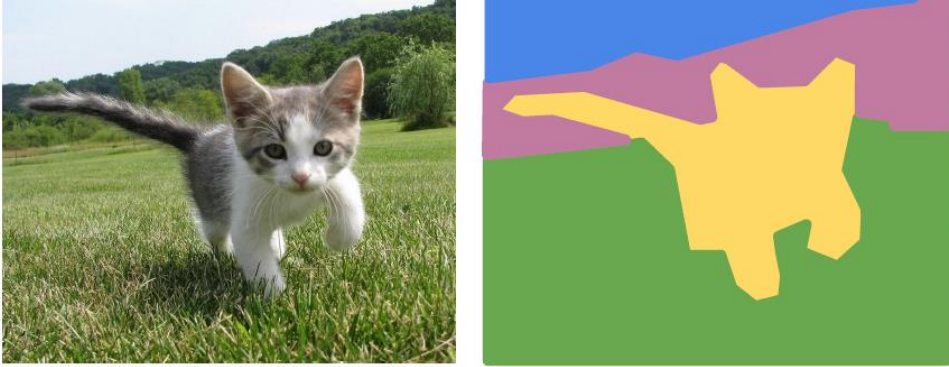
- Minaee et al. (2021). Image segmentation using deep learning: A survey. IEEE Trans. on Pattern Analysis and Machine Intelligence, 44(7), 3523-3542.
- Szeliski (2022), Chapter 6.4.
- Zhang, Lipton, Li and Smola (2023), *Dive into Deep Learning*, Chapter 14.9-14.11.
- Stanford University CS231n (2023): Deep Learning for Computer Vision. Lecture 11.

More classical approaches:

- Forsyth & Ponce (2012). Chapter 9.
- Sonka, Hlavac & Boyle (2015). Chapter 6 and 7.

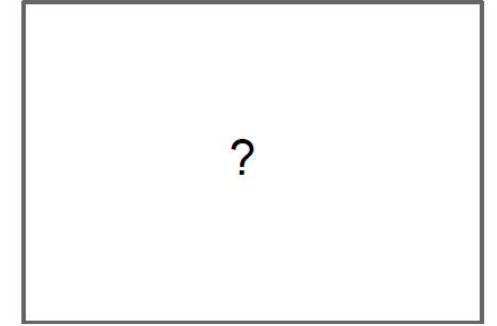
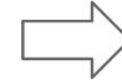
Next slides are mainly based on those from [cs231n](#)

Semantic Segmentation: the problem



GRASS, CAT,
TREE, SKY, ...

Paired training data: for each training image, each pixel is labeled with a semantic category.



At test time, classify each pixel of a new image.

We want to predict a mask
(where the object of interest is present).

If we just have one type of object → binary mask
(0: background; 1: object)

Image Segmentation = Pixel-level Classification

When you perform the prediction for every pixel → dense prediction

Semantic Segmentation: the problem

Full image



Can we perform segmentation using a single pixel?



Semantic Segmentation: the problem

Full image



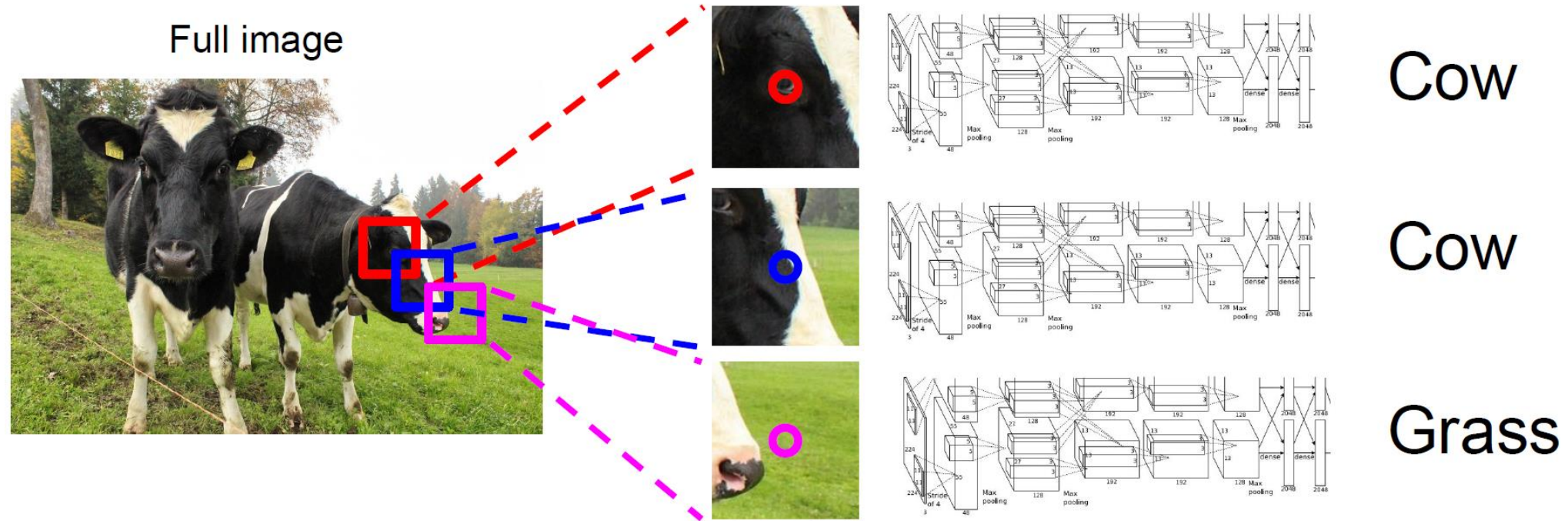
Can we perform segmentation using a single pixel?

Impossible to classify without any context

What context and how do we include it?

Semantic Segmentation

Solution 1



Extract patches/regions of fixed size and use a ConvNet to classify the central pixel!

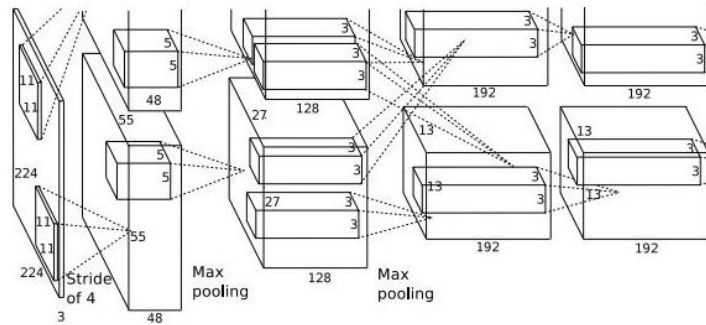
Problems:

- Very inefficient! We treat every patch independently.
- How to select patch size?

Semantic Segmentation

Solution 2

Full image



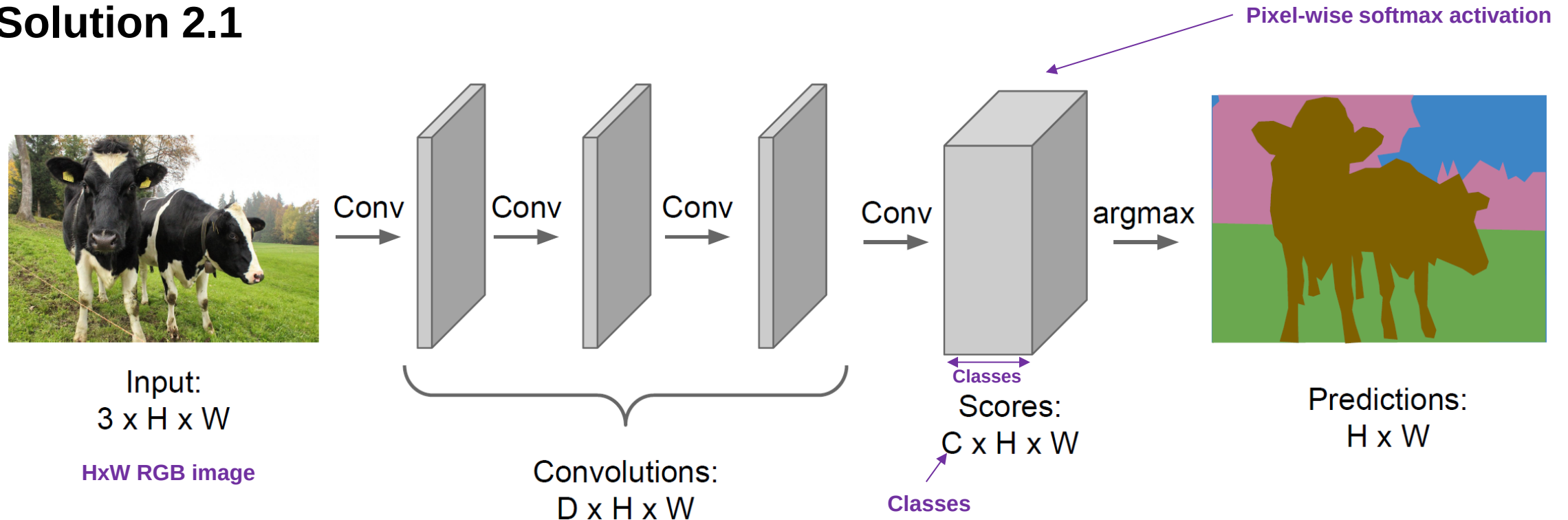
Encode the entire image with ConvNet, and do semantic segmentation on top!

Problem:

- classification architectures often reduce feature spatial sizes to go deeper, but **semantic segmentation requires the output size to be the same as input size**

Semantic Segmentation

Solution 2.1



Design a network with only **convolutional layers without downsampling operators** to make predictions for pixels all at once!!

Problem:

- convolutions at original image resolution will be **very expensive** (memory, number of operations,...)

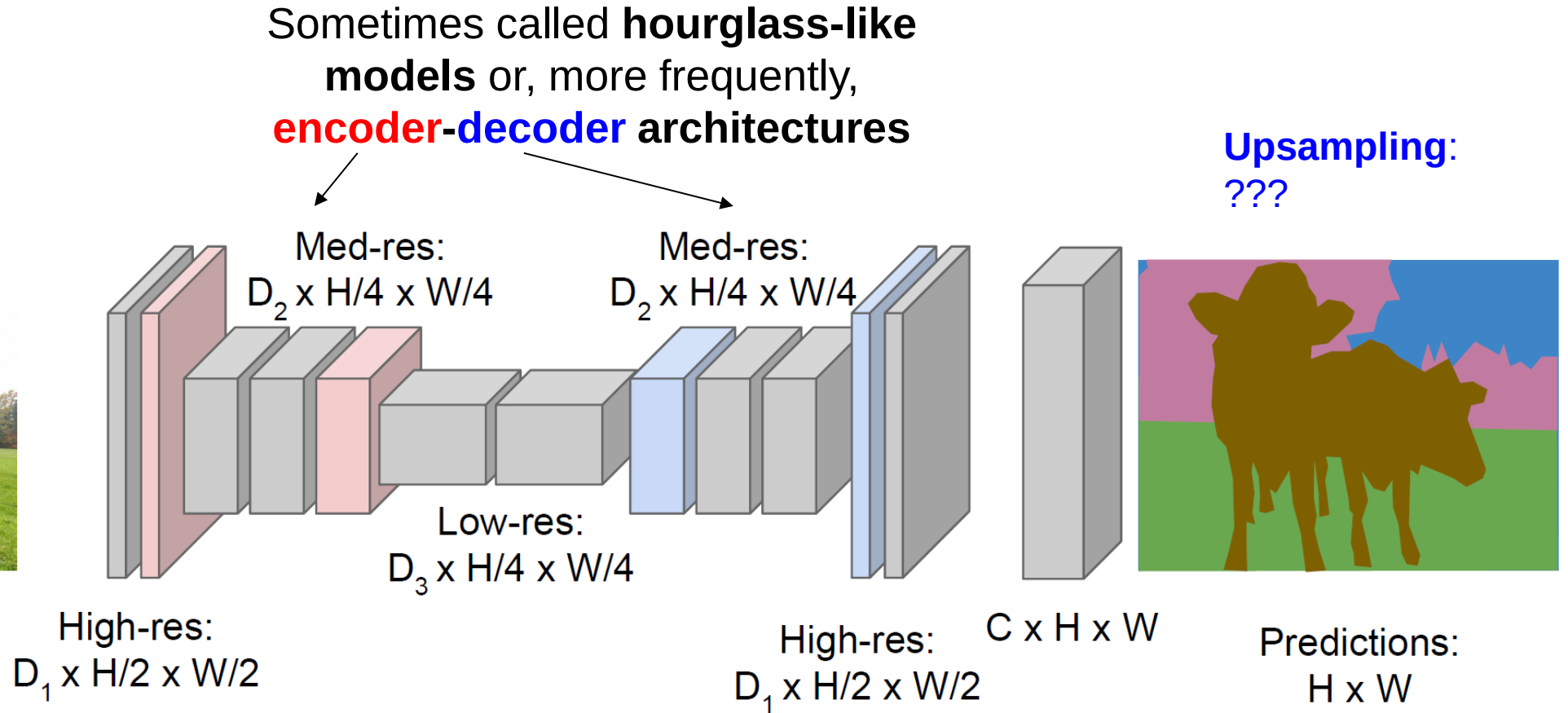
Fully Convolutional Neural Networks

Solution 2.2

Downsampling:
Pooling, strided
convolution



Input:
 $3 \times H \times W$



Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!

Fully Convolutional Neural Networks

In-Network Upsampling: “unpooling”

Nearest Neighbor

1	2
3	4



1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Input: 2 x 2

Output: 4 x 4

We simply repeat every element

“Bed of Nails”

1	2
3	4



1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

Input: 2 x 2

Output: 4 x 4

We simply place the value in a particular position in the output, filling the rest with zeros

Fully Convolutional Neural Networks

In-Network Upsampling: “max unpooling”.

Smarter “bed of nails” method. Rather than a predetermined/fixed location for the “nails”, we use the position of the maximum elements from the corresponding max pooling layer earlier in the network.

Max Pooling

Remember which element was max!

1	2	6	3
3	5	2	1
1	2	2	1
7	3	4	8

Input: 4 x 4



5	6
7	8

Output: 2 x 2



...

Rest of the network

Max Unpooling

Use positions from pooling layer

1	2
3	4

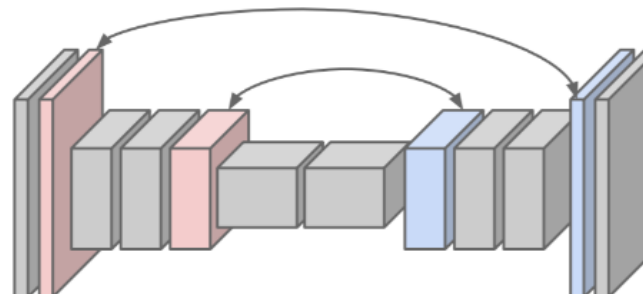
Input: 2 x 2



0	0	2	0
0	1	0	0
0	0	0	0
3	0	0	4

Output: 4 x 4

Corresponding pairs of
downsampling and
upsampling layers

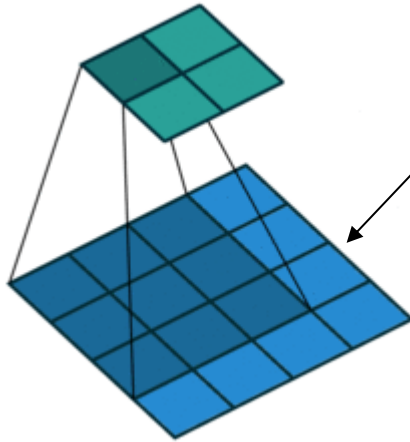


Fully Convolutional Neural
Networks are often
symmetric!!!

Fully Convolutional Neural Networks

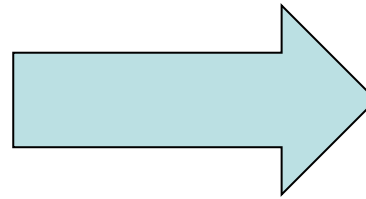
Learnable Upsampling: Transposed Convolution.

Convolution



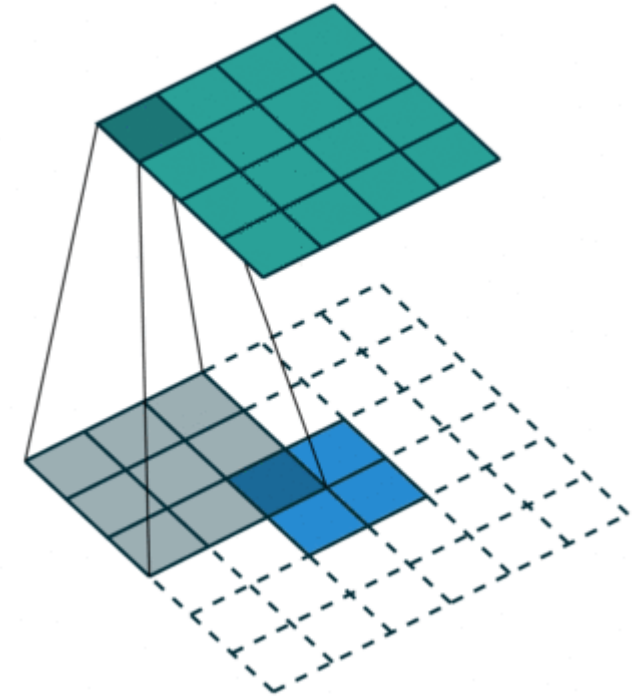
Convolution of a 3×3 kernel on a 4×4 input with unitary stride and no padding.

The transposed convolution can be considered as the operation that allows to **recover the shape (not the input values) of the initial feature map**.



Blue maps are inputs, and *cyan* maps are outputs.

Transposed convolution



The transpose will then have an output of shape 4×4 when applied on a 2×2 input.

https://github.com/vdumoulin/conv_arithmetic

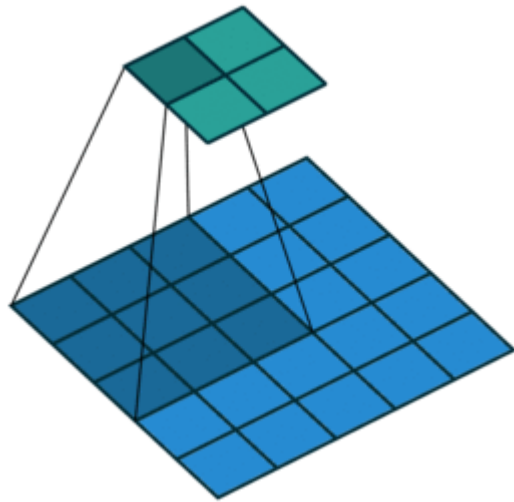
Dumoulin, Vincent, and Francesco Visin. "A guide to convolution arithmetic for deep learning." *arXiv preprint arXiv:1603.07285* (2016).

You could also use, for instance, upsampling via bilinear interpolation, but experiments show that **it's commonly better to learn to upsample**. See Long et al., "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015

Fully Convolutional Neural Networks

Learnable Upsampling: Transposed Convolution.

Convolution

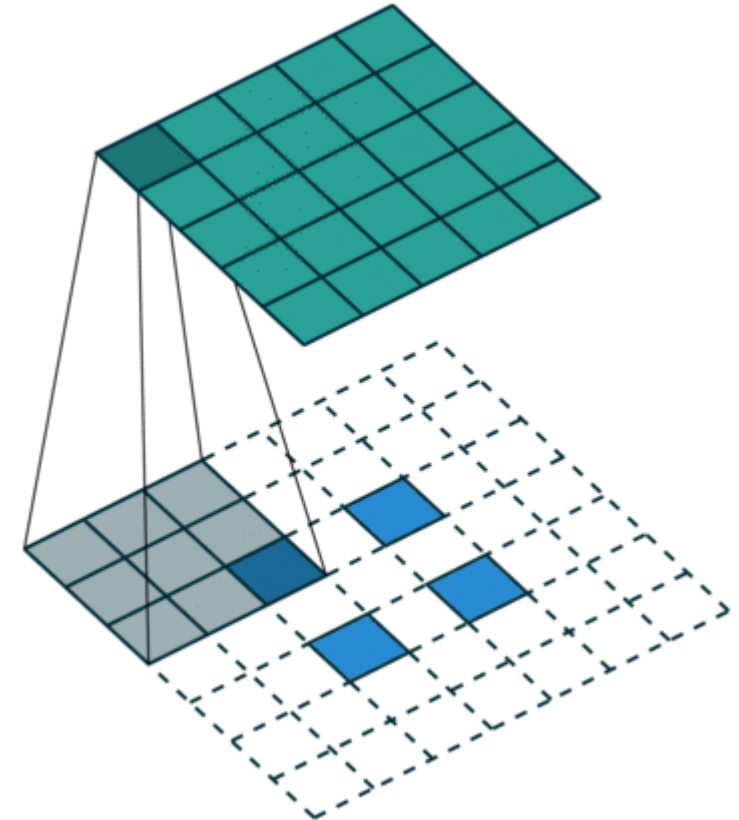


Convolution of a 3×3 kernel on a 5×5 input with 2×2 stride and no padding.



Blue maps are inputs, and *cyan* maps are outputs.

Transposed convolution

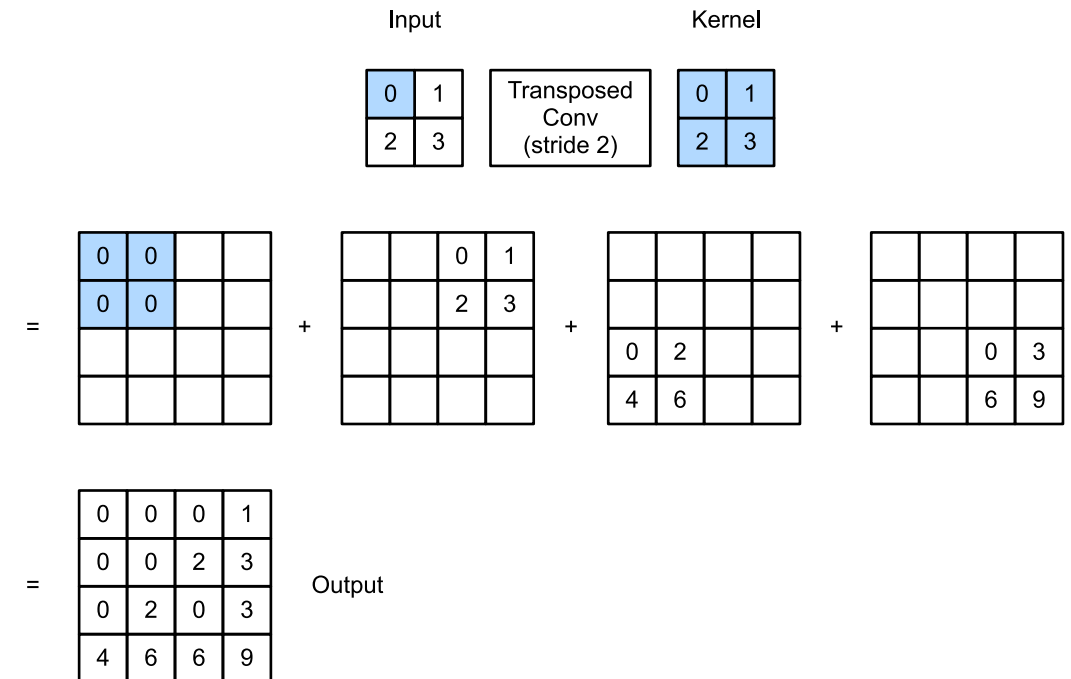
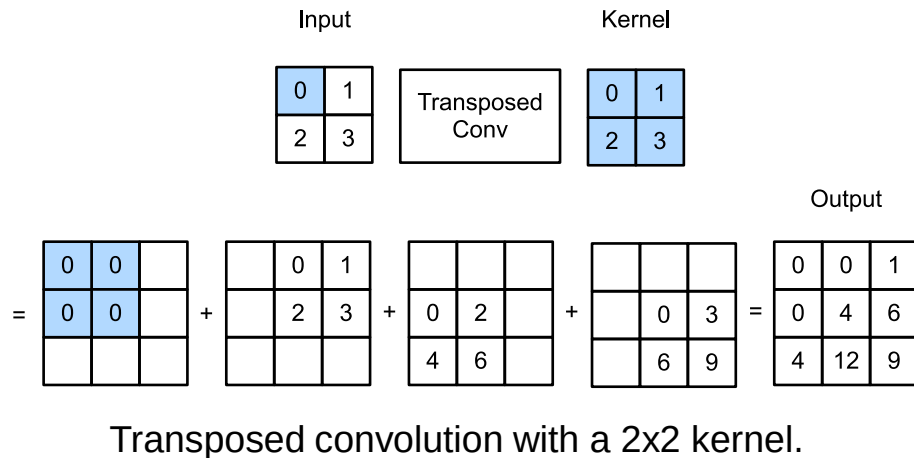


The transpose will then have an output of shape 5×5 when applied on a 2×2 input.

(We convolve a 3×3 kernel over a 2×2 input (with zeros inserted between inputs) padded with a 2×2 border of zeros using unit strides)

Fully Convolutional Neural Networks

Learnable Upsampling: Transposed Convolution.



Transposed convolution with a 2x2 kernel with stride 2.

https://d2l.ai/chapter_computer-vision/transposed-conv.html

The transposed convolution is not the inverse of a convolution, and thus *deconvolution* does not seem a suitable name for the operation. For deconvolution, see Shree Nayar's "First Principles in Computer Vision"

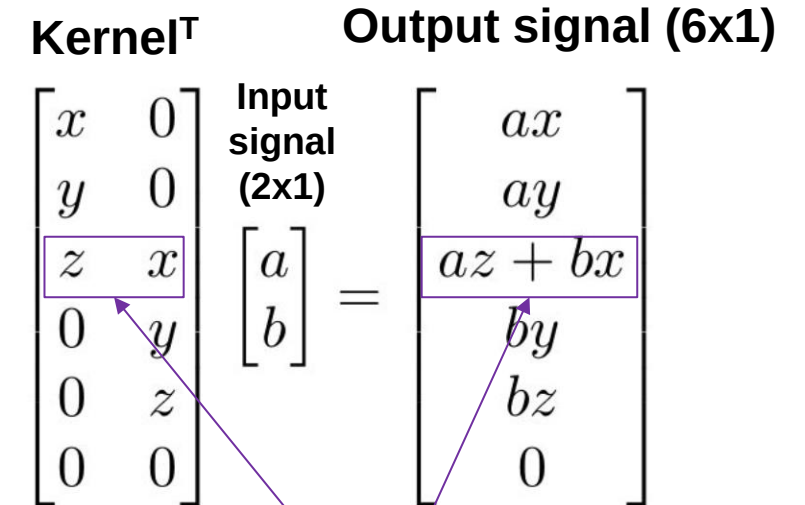
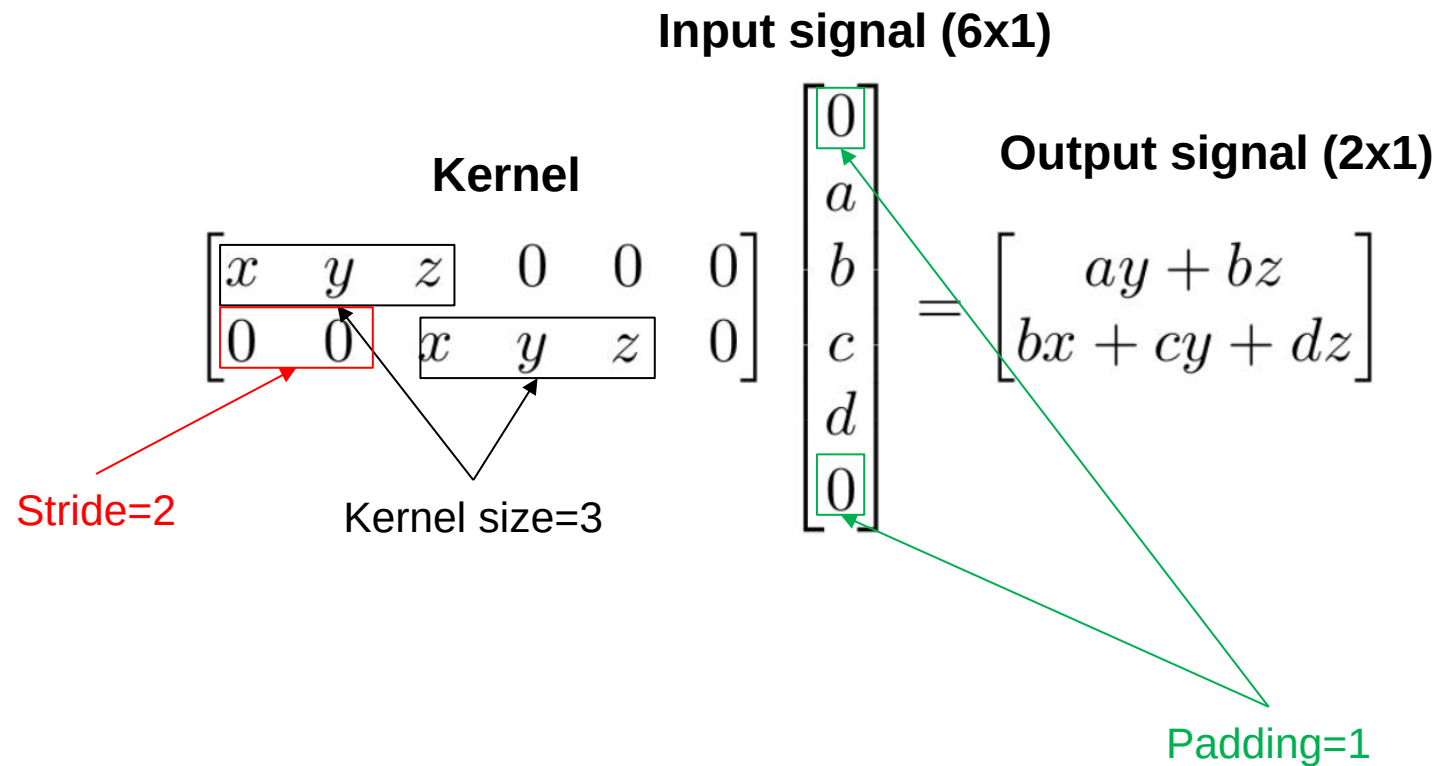
Fully Convolutional Neural Networks

Learnable Upsampling: Transposed Convolution.

Why is it called transposed convolution???

We can express convolution in terms of a matrix multiplication. 1D Example:

Transposed convolution multiplies by the transpose of the same matrix.



For filter sizes which produce an overlap in the output feature map, the overlapping values are simply added together.

Fully Convolutional Neural Networks

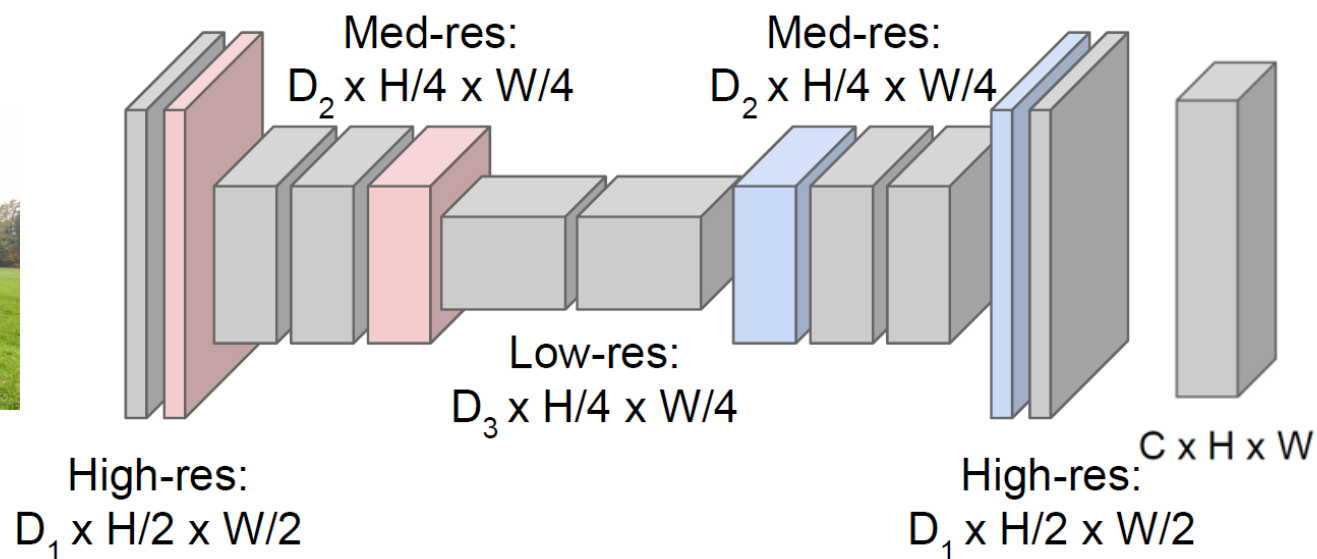
Downsampling:
Pooling, strided
convolution

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!

Upsampling:
Unpooling or strided
transposed convolution



Input:
 $3 \times H \times W$



Predictions:
 $H \times W$

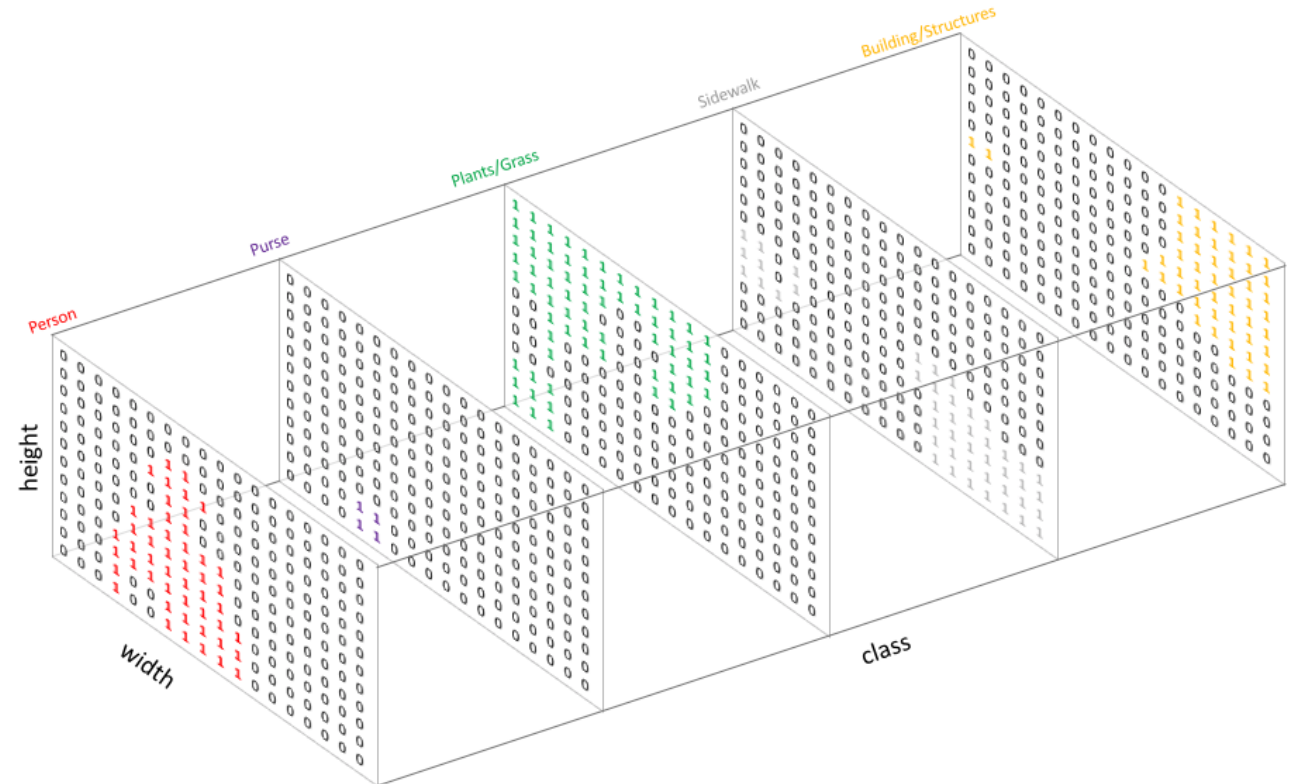
Fully Convolutional Neural Networks

What about the target labels employed for training?



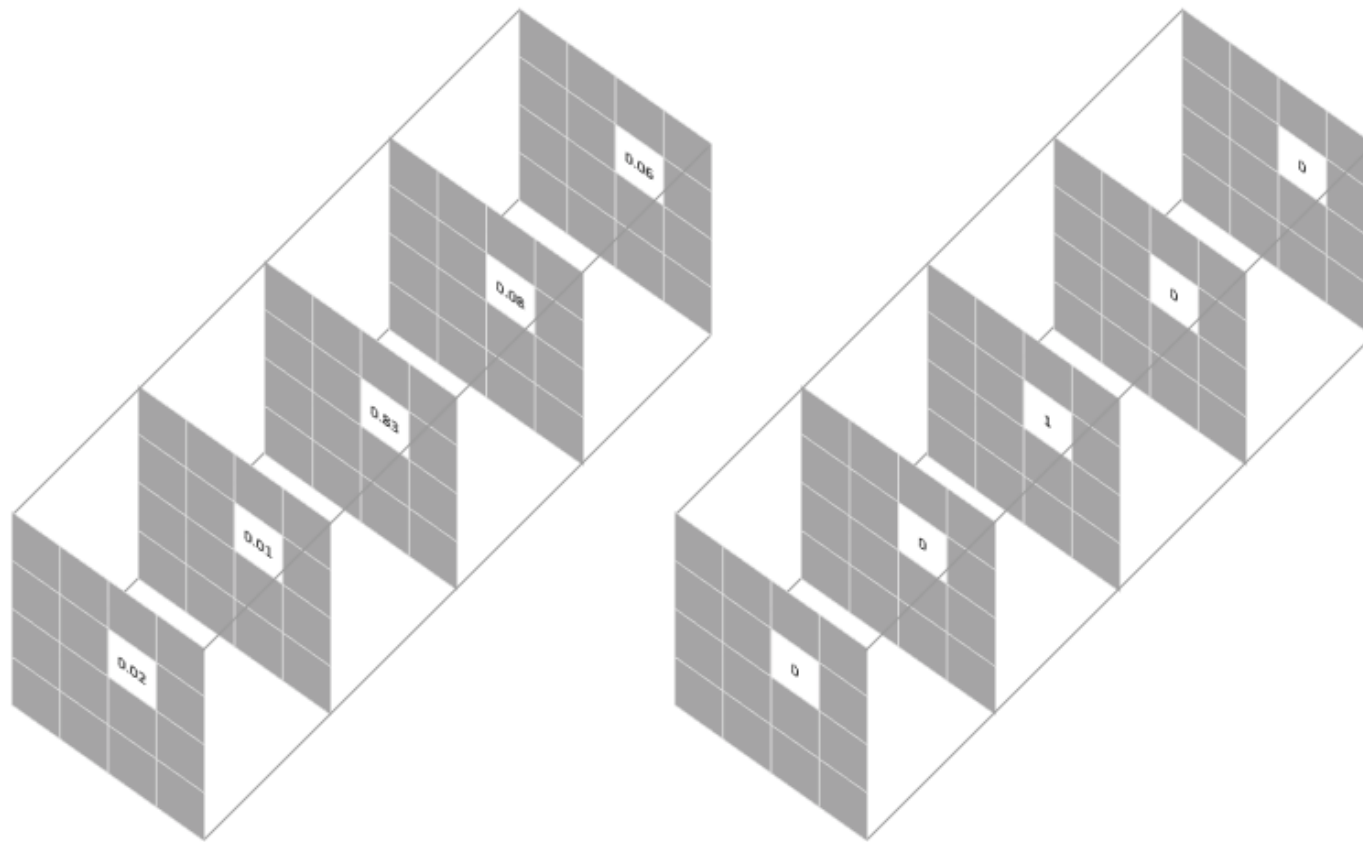
0: Background/Unknown
1: Person
2: Purse
3: Plants/Grass
4: Sidewalk
5: Building/Structures

Our target is the one-hot encoding of the class labels (we create an output channel for each possible class)



We can train our segmentation model using a **pixel-wise cross-entropy loss** (having a softmax per pixel).

Fully Convolutional Neural Networks



Pixel-wise loss is calculated as the log loss, summed over all possible classes

$$-\sum_{\text{classes}} y_{\text{true}} \log(y_{\text{pred}})$$

This scoring is repeated over all **pixels** and averaged

We're assuming equal learning to each pixel in the image. This can be a **problem if your classes have unbalanced representation in the image**, as training can be dominated by the most prevalent class.

Possible solution:

- weighting strategy: enhance importance of minority classes

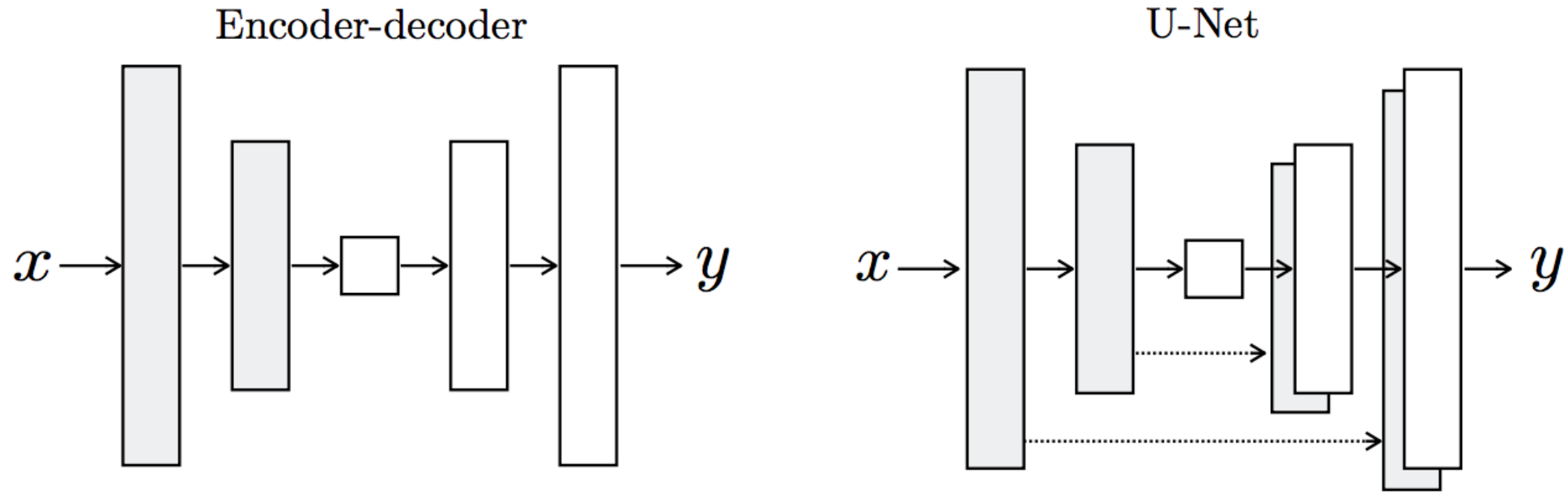
Sugino, Takaaki, et al. "Loss weightings for improving imbalanced brain structure segmentation using fully convolutional networks." *Healthcare*. Vol. 9. No. 8., 2021.

Other popular segmentation losses are also available, like the [Dice loss](#) or the [Focal loss](#).

Image Classification vs Image Segmentation

- In Image Classification feature resolution decreases as we go deeper into the network.
 - Help reduce computational cost (memory, operations) while preserving transformation invariance.
- But in Image Segmentation spatial information is necessary.
 - **As we go deeper into the network, feature maps become increasingly good at encoding “what is in the image?” but information about “where in the image?” is lost.**
- Let's see one strategy to deal with this: U-Net

U-Net



[Image Source](#)

U-Net

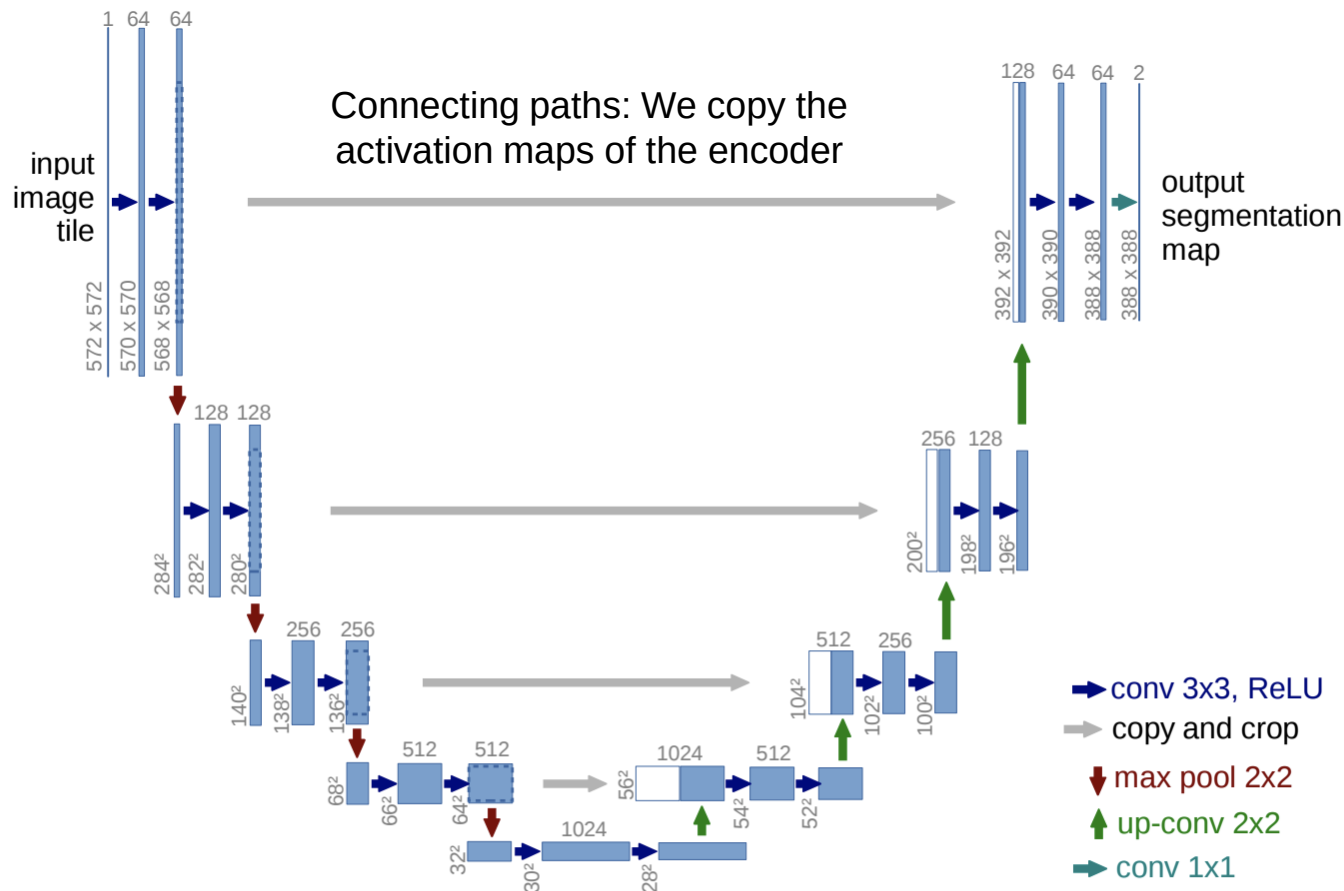
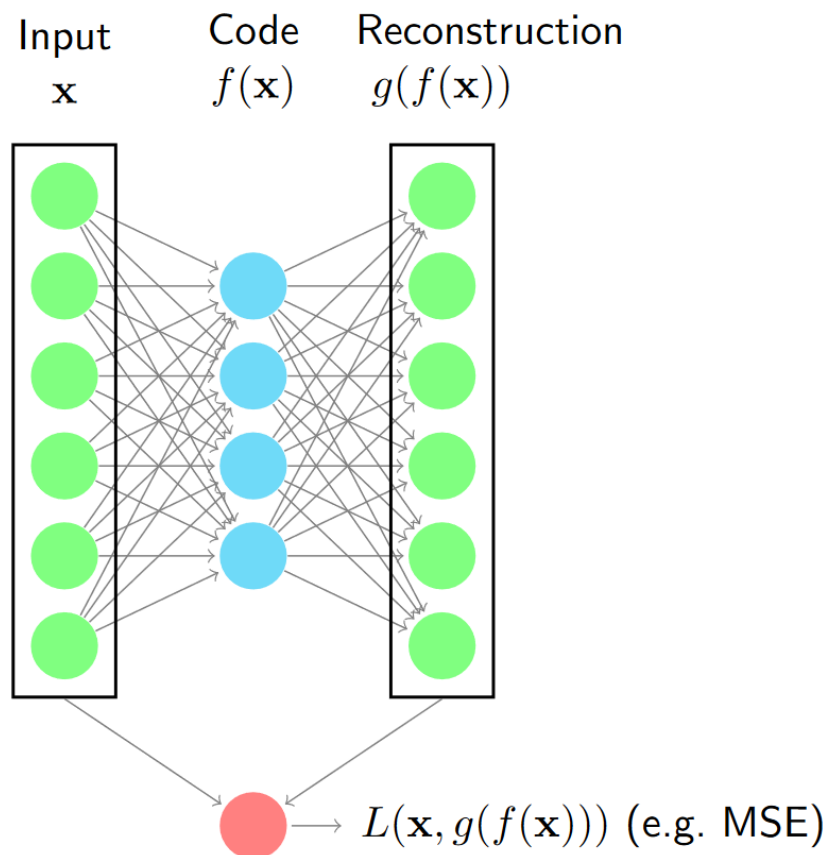


Fig. 1. U-net architecture (example for 32x32 pixels in the lowest resolution). Each blue box corresponds to a multi-channel feature map. The number of channels is denoted on top of the box. The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations.

- ConvNet with an encoder-decoder architecture (U-shape).
- Thanks to the connecting paths, we combine features from the **encoder** (details, **spatial information**) and the **decoder** (high-level **semantic information**).
- Successful when training with few images.
- It's also used in image super-resolution and image generation.
 - Many generative models (Stable Diffusion, DALL-E 2,...) use this type of U-Net architecture under the hood.

Autoencoders

- These encoder-decoder architectures can be used for many different tasks, like image compression and denoising.



We want to learn an efficient encoding of input data.

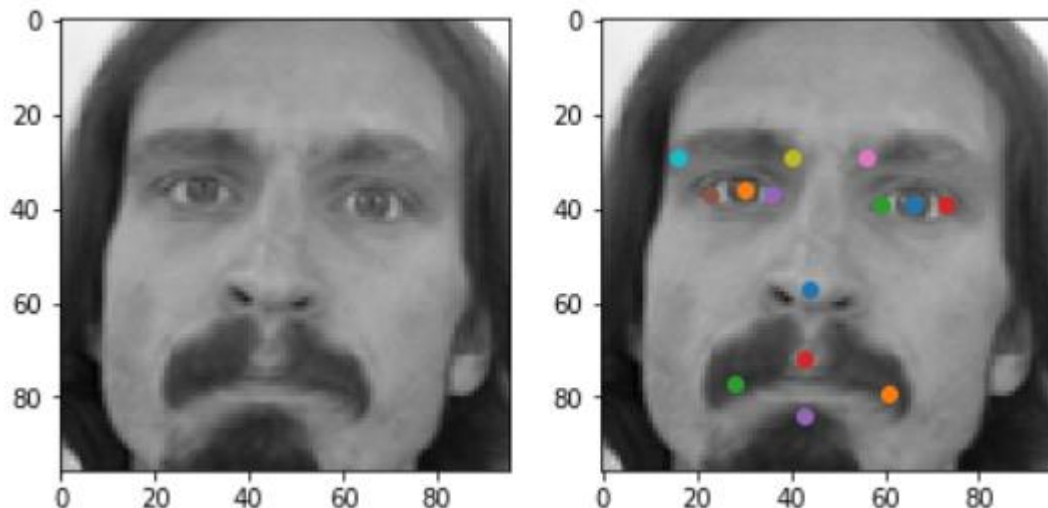


Denoising autoencoders:

- We corrupt input data on purpose, adding noise or masking some of the input values.
- The model is trained to predict the original, uncorrupted data.
- The autoencoder must undo this corruption.

Keypoint Regression

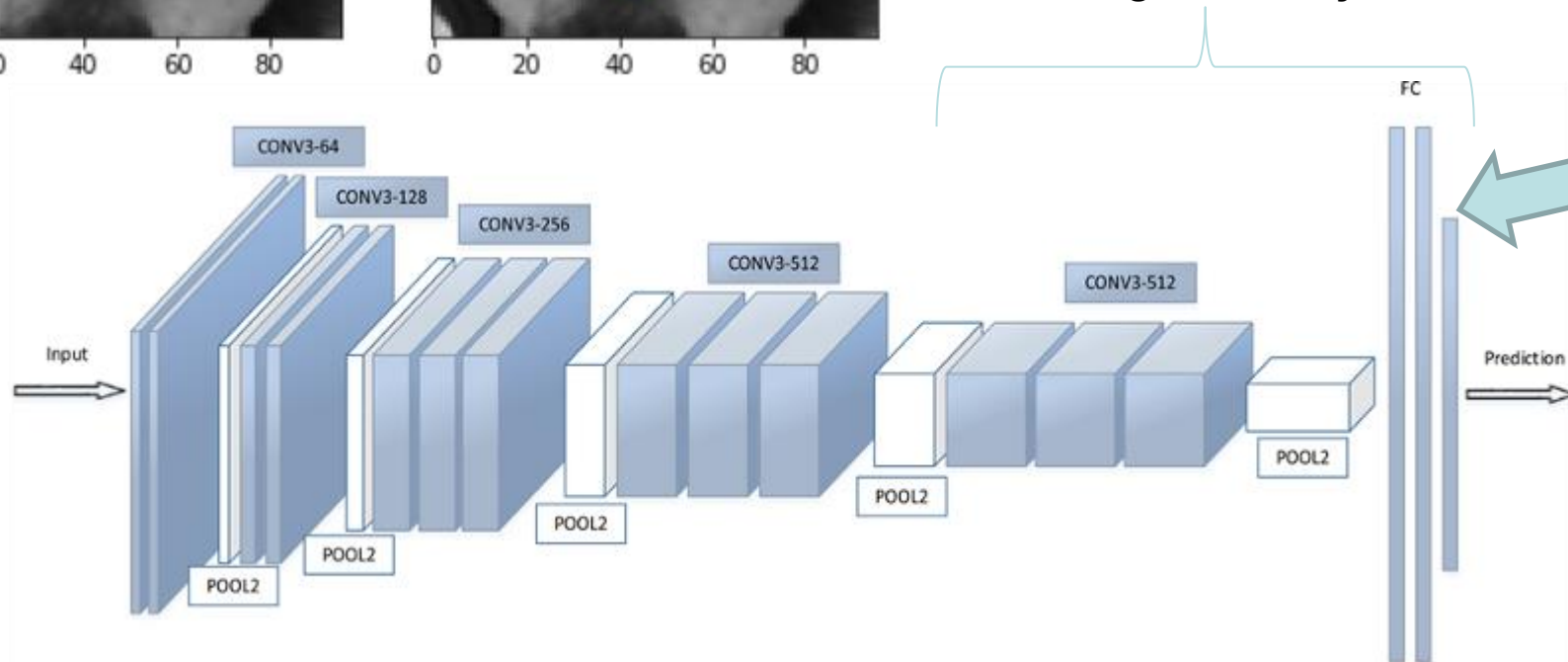
We want to locate keypoints in images (e.g., in faces)



Option 1:

Directly predict Cartesian coordinates (x,y) for each keypoint

Fine-tuning of last layers

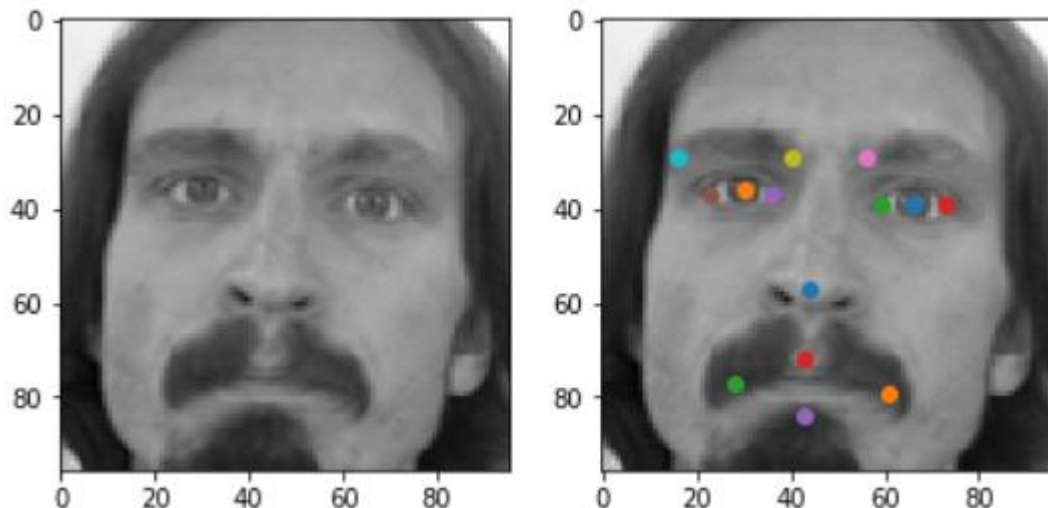


FC1000 is replaced by BN + a regression layer

We are forcing the network to map from the image to Cartesian coordinates. Maybe too complex. Likely to overfit.

Keypoint Regression

We want to locate keypoints in images (e.g., in faces)

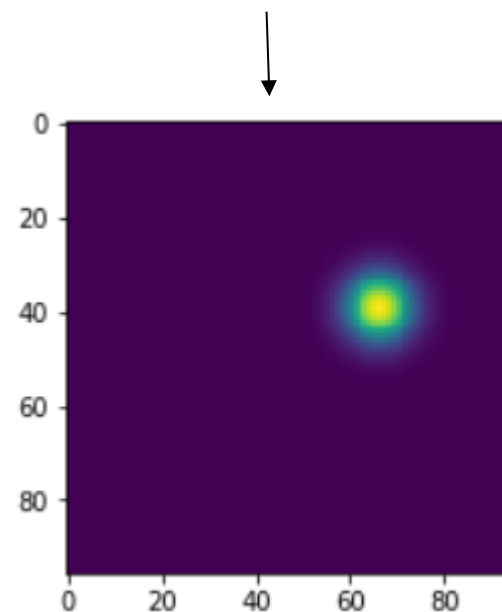


Heatmaps generally work better than direct joint/keypoint regression.

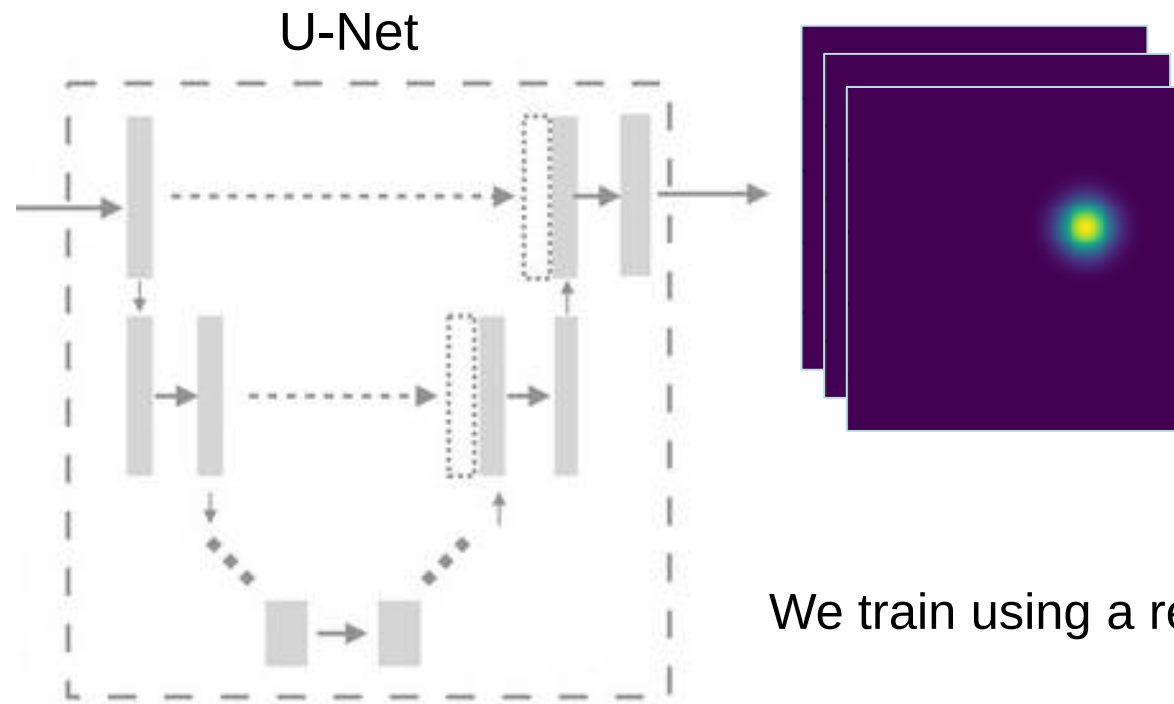
Option 2:

Perform heatmap regression for each keypoint.

Each keypoint is represented by a heatmap (smooth version of our Cartesian coordinates), that represents a wider range of “correct” answers (likelihood of a specific keypoint residing at that pixel). Similar to label smoothing to make our models less (over)confident.



Keypoint Regression



We train using a regression loss (MSE).

If we want to locate 3 keypoints in a $H \times W$ image:

- **output: $3 \times H \times W$ tensor**, with **each channel representing one of our keypoints/classes**
- **each pixel would represent the likelihood** of that pixel being the keypoint we're looking for
- to **retrieve the keypoints location**, we find the location of the highest activation in each output heatmap

DeepLabV3+

Xception backbone (depthwise separable convolutions)

+ atrous convolutions (larger receptive field, no increase in computational cost)

+ atrous spatial pyramid pooling (ASPP) (for multi-scale feature extraction)

+ encoder-decoder architecture (with BN, ReLU and 1x1,3x3 convolutions)

Check <https://learnopencv.com/deeplabv3-ultimate-guide/> for details

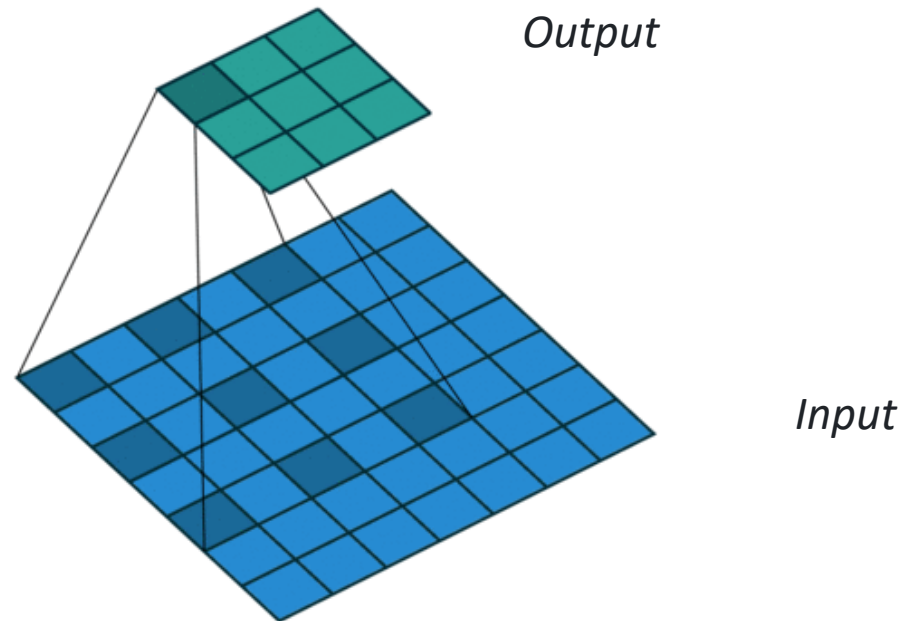
DeepLabV3+

Xception backbone (depthwise separable convolutions)

+ **atrous convolutions** (larger receptive field, no increase in computational cost)

+ atrous spatial pyramid pooling (ASPP) (for multi-scale feature extraction)

+ encoder-decoder architecture

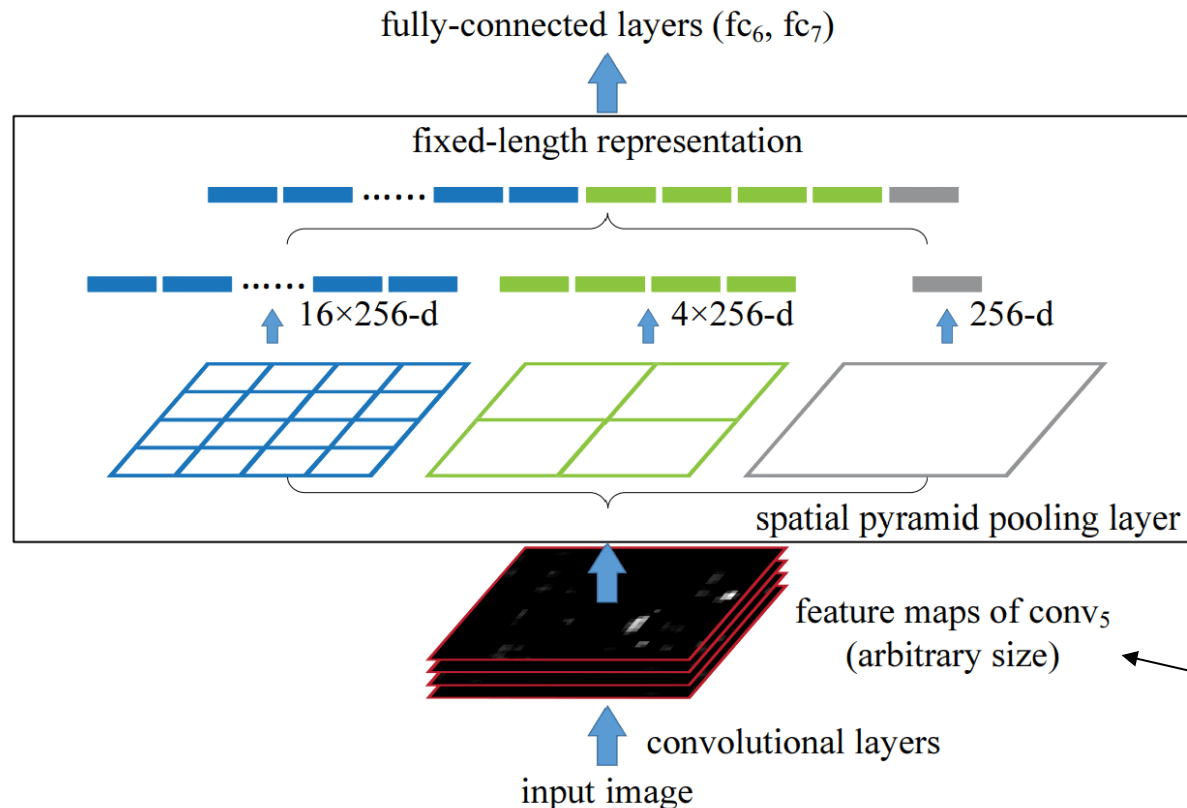


DeepLabV3+

Xception backbone (depthwise separable convolutions)

+ atrous convolutions (larger receptive field, no increase in computational cost)

+ atrous **spatial pyramid pooling** (ASPP) (for multi-scale feature extraction)



Effective to resample features at different scales.

Provides a fixed-length representation, independent of the input image size.

→ GlobalAveragePooling and GlobalMaxPooling represent another way of doing it!

Different pooling sizes (from right to left):
global, 4 bins, 16 bins

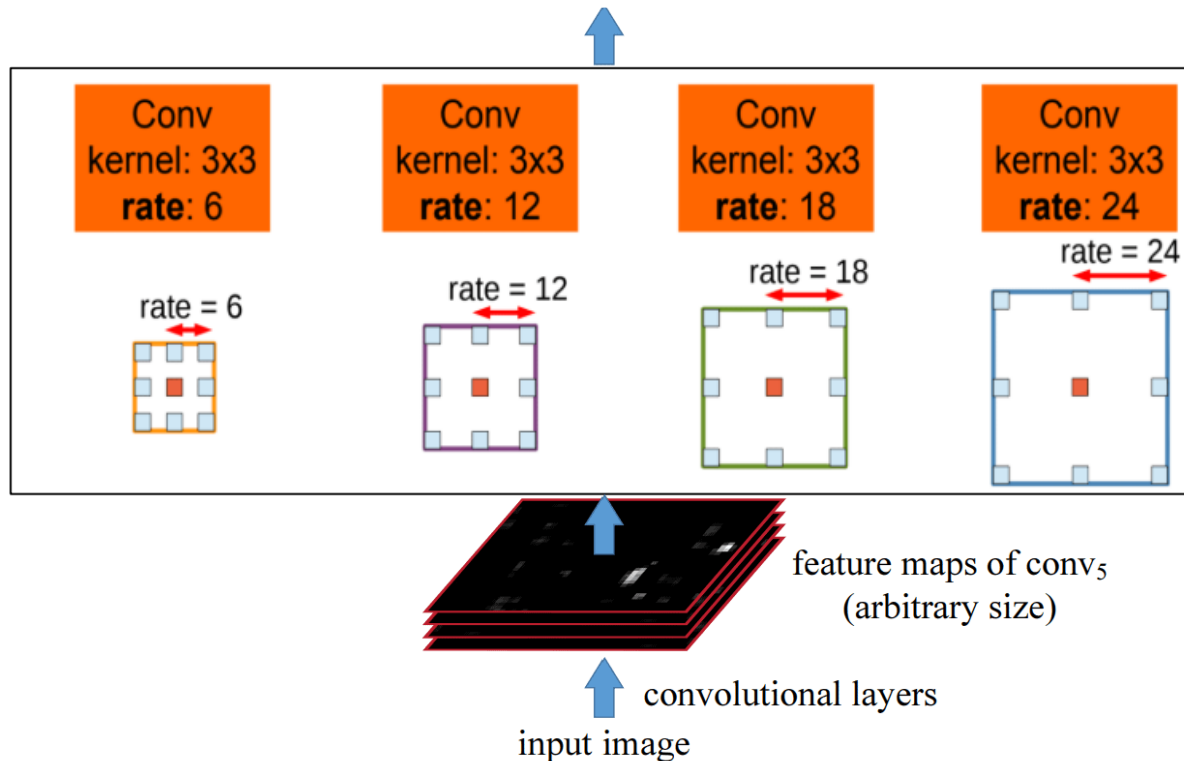
256 filters / activation maps

DeepLabV3+

Xception backbone (depthwise separable convolutions)

+ atrous convolutions (larger receptive field, no increase in computational cost)

+ **atrous spatial pyramid pooling (ASPP)** (for multi-scale feature extraction)

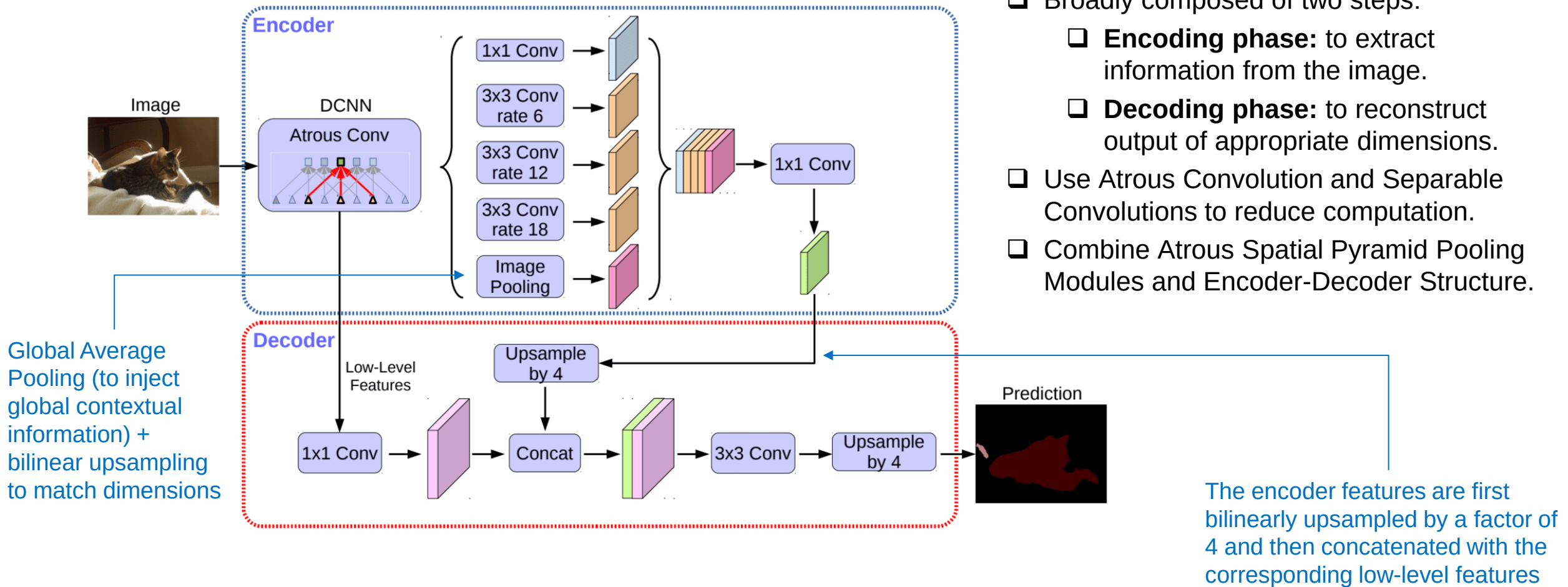


Multiple atrous convolutional layers in parallel → We capture objects and image context at multiple scales.

ASPP = **SPP** but using atrous convolutions with different dilation rates

DeepLabV3+

Full encoder-decoder architecture



How do we evaluate segmentation?

- Accuracy: percentage of pixels correctly classified/segmented
 - Poor metric. E.g., if objects of interest only represent 10% of pixels in the image, and we systematically segment everything as background → acc=90%

- Dice Similarity Coefficient (DSC) or F1 score:

$$\text{DSC} = \frac{2|X \cap Y|}{|X| + |Y|} = \frac{2TP}{2TP + FP + FN} \in [0,1]$$

Number of segmented pixels Number of pixels belonging to the ground truth TP: true positives | FP: false positives | FN: false negatives

- You have one metric value per class. If you want a single overall value, you'd compute the mean of all these DSC values.

How do we evaluate segmentation?

- Dice Similarity Coefficient (DSC) or F1 score :

$$DSC = \frac{2|X \cap Y|}{|X| + |Y|} = \frac{2TP}{2TP + FP + FN} \in [0,1]$$

↑ Segmented pixels ↑ Ground truth

TP: true positives | FP: false positives | FN: false negatives

- Jaccard Index (JI) or Intersection over Union (IoU): $JI = \frac{|X \cap Y|}{|X \cup Y|} = \frac{DSC}{2 - DSC}$

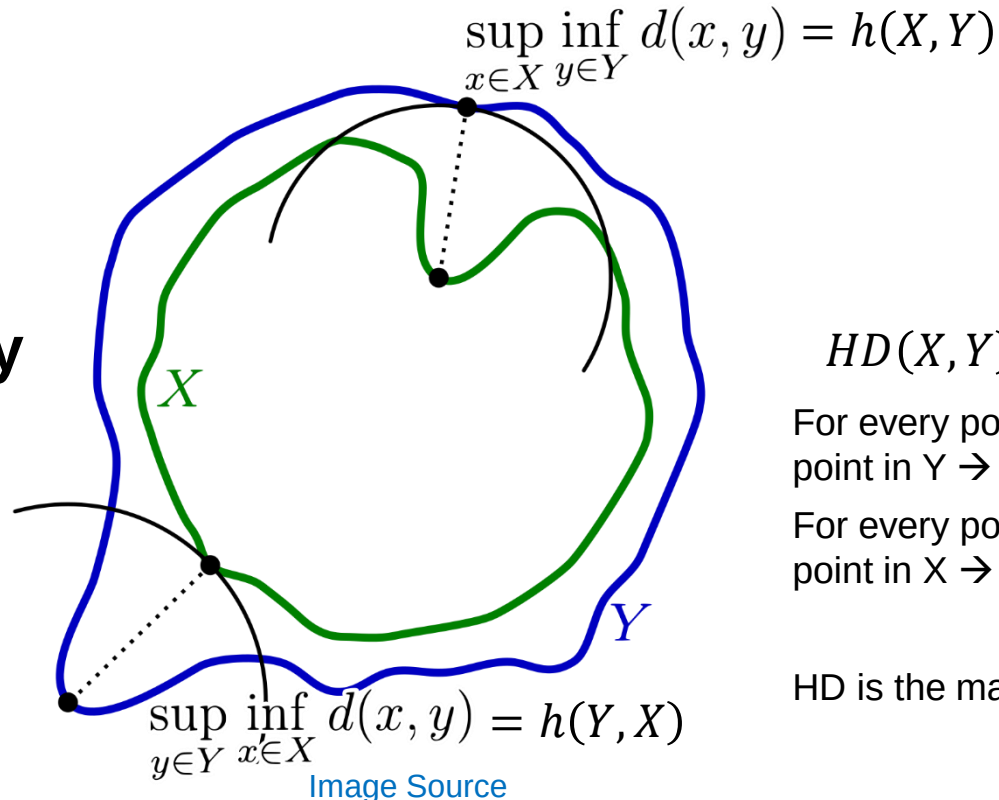
<https://giou.stanford.edu/>



How do we evaluate segmentation?

- Hausdorff Distance (HD):
 - largest of all distances from any point in the boundary of the resulting segmentation to the closest point in the ground truth.

- we try to measure how close the contours are to each other. This is a good metric if **boundary location accuracy** is particularly relevant.



$$HD(X, Y) = \max(h(X, Y), h(Y, X))$$

For every point in X, compute min distance to any point in Y → take the max of these values: $h(X, Y)$

For every point in Y, compute min distance to any point in X → take the max of these values: $h(Y, X)$

HD is the max of these two distances.

Image Segmentation

Pablo Mesejo

pmesejo@go.ugr.es

Universidad de Granada

Departamento de Ciencias de la Computación e Inteligencia Artificial



UNIVERSIDAD
DE GRANADA



DaSCI

Instituto Andaluz de Investigación en
Data Science and Computational Intelligence